

Technical Specification Document

Tram Name: Natural Disaster

Team Members:

Dongyun Lee

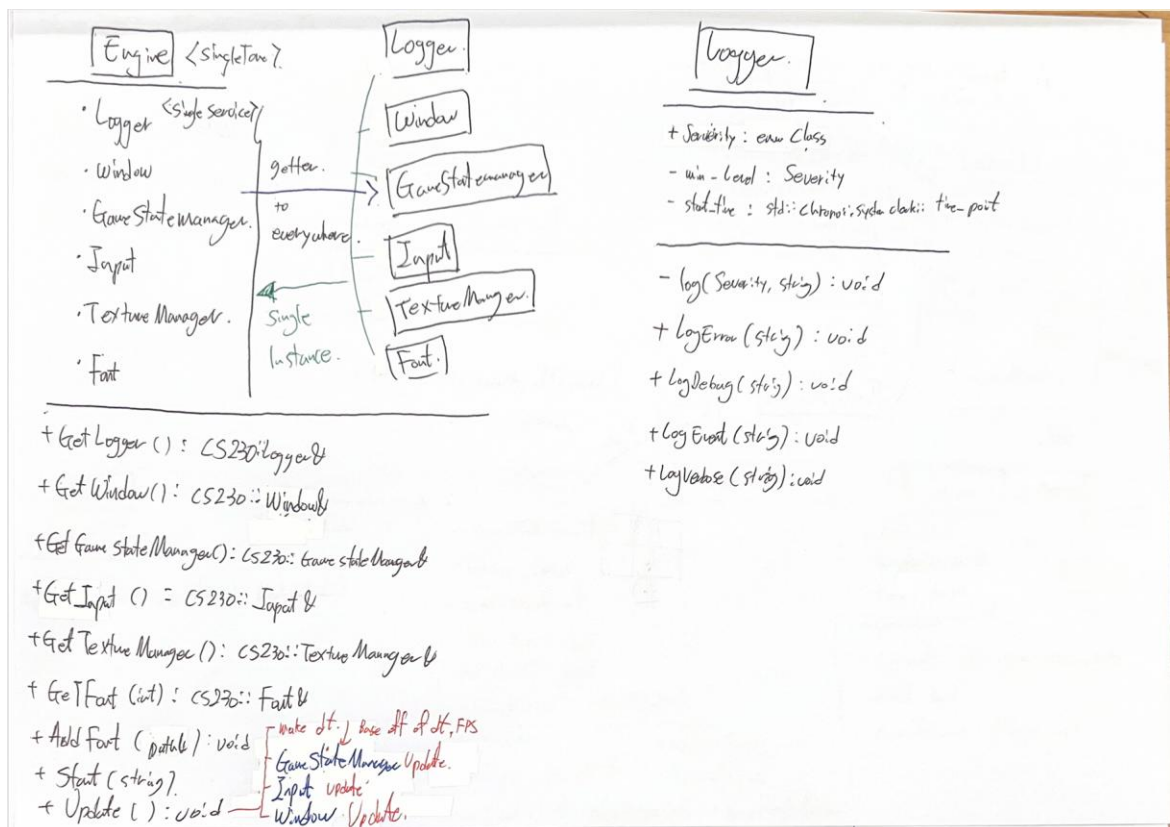
Taekyung Ho

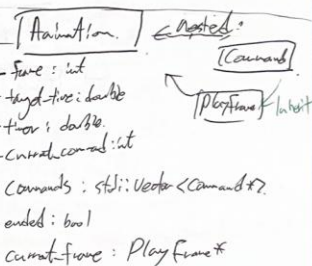
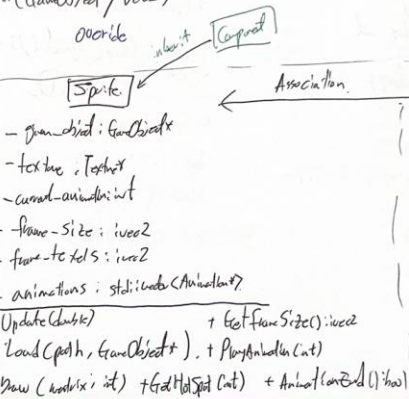
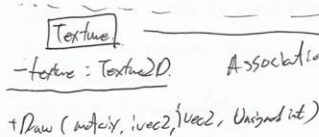
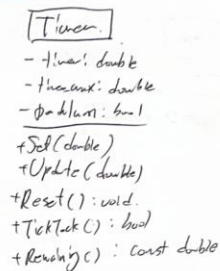
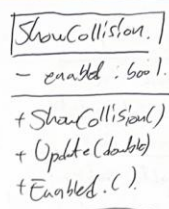
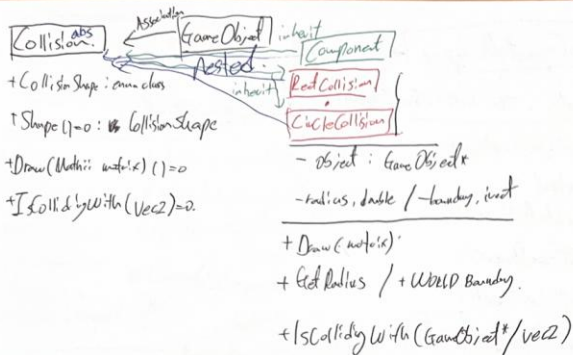
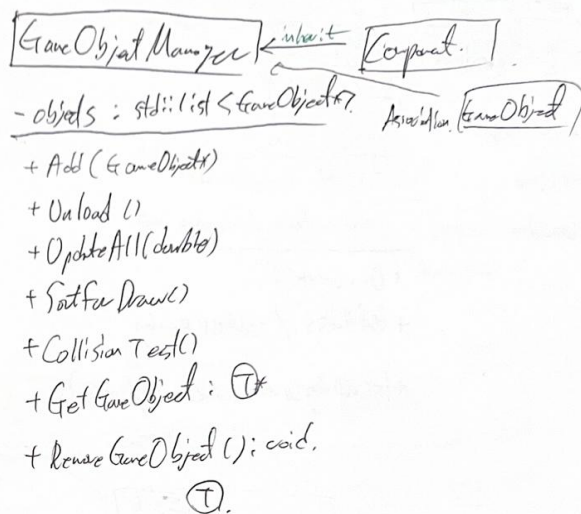
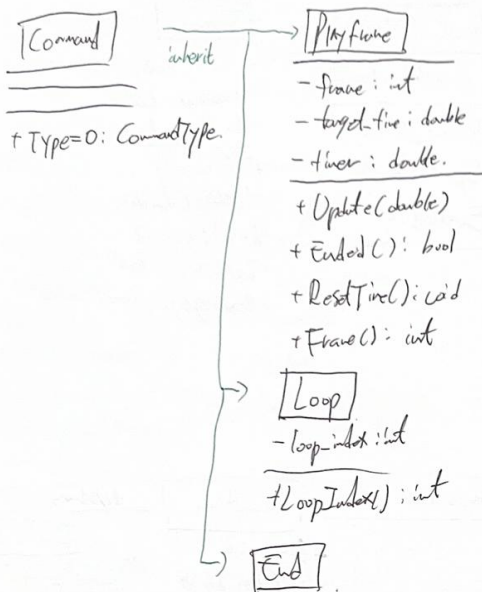
Sanghyuk Kim

Youngwoo Lee

Game Name: 10..9..8..

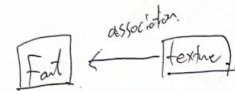
1. Provide a diagram to show all the classes in your game engine and how they interact with each other using arrows. Label the classes and label the arrows, as well, to describe the interactions. (If you need help, try looking into UML documentation standards.) (10 pts)





Window

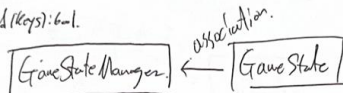
- size : Math::ivec2.
- + Update() : void
- + GetSize() : Math::ivec2.
- + Clear (unsigned int) : void.



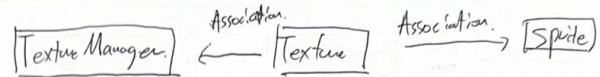
- original_image : Image
- texture : Texture.
+ PrintToTexture (string, unsigned int) : Texture*

Input

- + keys : enum class
- + keys_down : std::vector<bool>.
- + previous_keys_down : std::vector<bool>.
- + Update() : void.
- + KeyDown(keys) : bool
- + KeyJustPressed(keys) : bool
- + KeyJustReleased(keys) : bool.

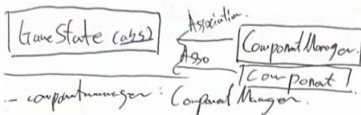


- Status : enum class
- game_states : std::vector<GameState*>
- current_gamestate : GameState*
- next_gamestate : GameState*
+ Update(double) : void
+ AddGameState(GameState*) : void
+ SetNextGameState(int) : void
+ ClearNextGameState() : void
+ ReloadState() : void.
+ HasGameEnded() : bool

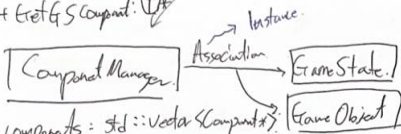


- textures :
- loaded_textures
+ Load(path) : Texture*
+ StartRenderTextureMode(int, int)
+ EndRenderTextureMode() : Texture*

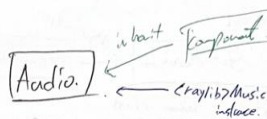
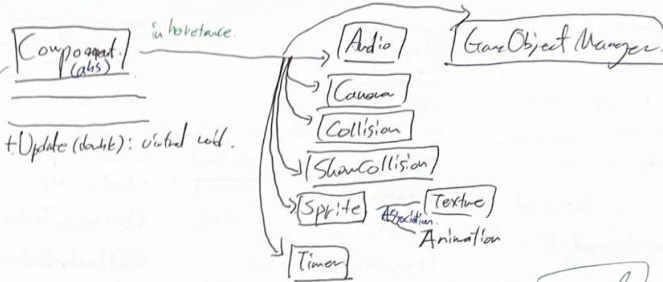
+ GetGSComponent() : T*



- componentmanager : ComponentManager.
+ Load = 0. # AddGSComponent(Component*)
+ Update(double) = 0. # UpdateGSComponents(double)
+ Unload() = 0. # RemoveGSComponent()
+ GetNext() = 0. # ClearGSComponents.
+ Draw() = 0.
+ CreateGSComponent() : T*



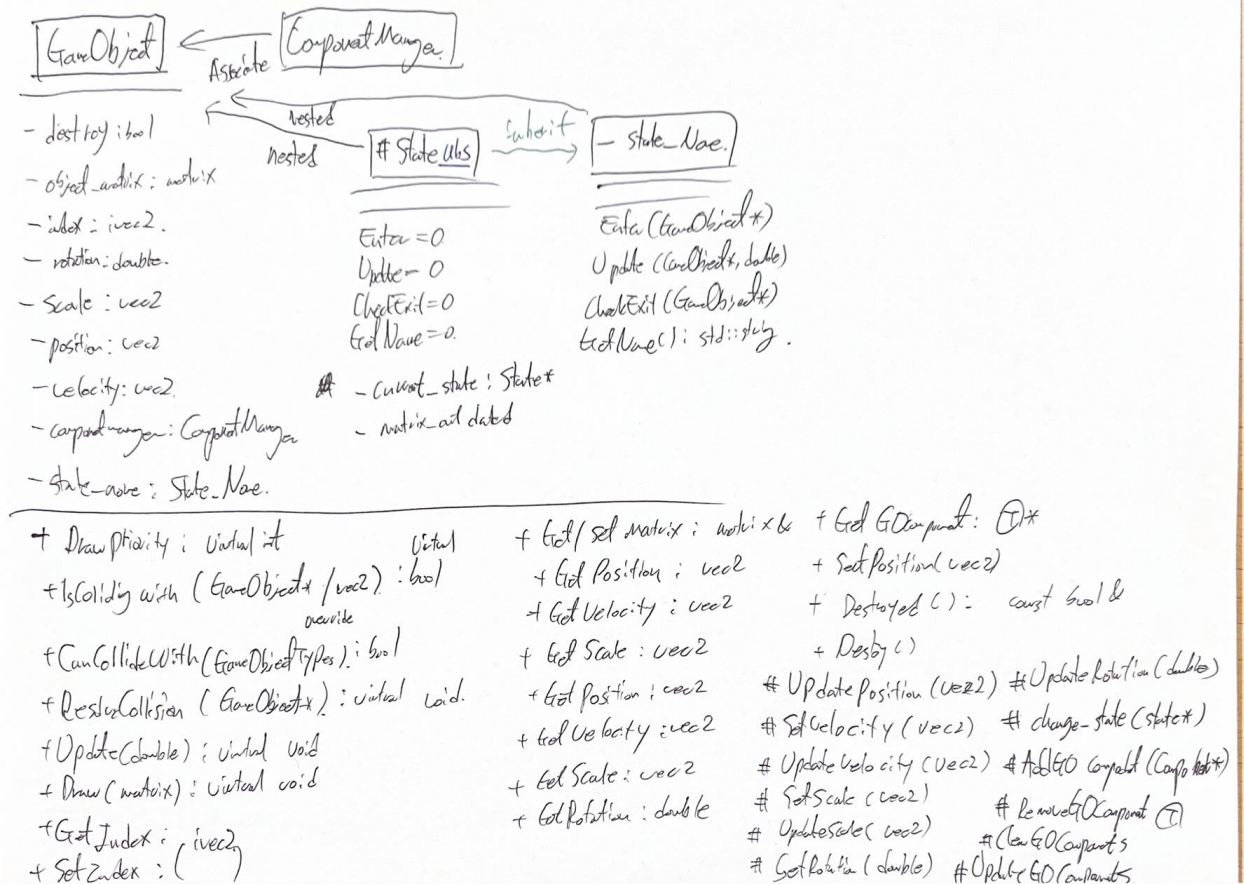
- components : std::vector<Component*>
+ UpdateAll(double)
+ AddComponent(component*)
+ GetComponent() : T*
+ RemoveComponent() : void
+ Clear() : void.



- music : Music
+ Play() : void.
+ Stop() : void
+ Update() : void
+ SetCoping(bool) : void.



- limit : Math::ivec2
- position : Math::vec2
- player - 2000 Math::rect
+ SetPosition(Math::ivec2)
+ SetLimit(Math::ivec2).
+ Update(Math::vec2)
+ GetPosition(const : const Math::vec2)
+ GetMatrix() : Math::TransformationMatrix.



2. List each Game Object Component and explain its purpose in your game.

-Timer

: enemies have to set timer after they finished their behaviors. And after time is over, enemies said turn-manager that they are ready to change turn.

-Sprite

: we use tile sheets for drawing map. So we use sprite's texel based drawing. And we also use sprite's animation function. In sprite, there is instance of texture.

-Texture

: if we don't have to use texel based drawing or animation, like ui or fonts, we just use texture class instead.

3. List each Game State Component and explain its purpose in your game.

- Game Object Manager

After making game objects in game, we put game objects in list of objects in game object manager. Because almost all objects are inherited from base object class, we can update and draw every objects by just calling update-all and draw-all function in game object manager. Also, we want to order objects in list of objects so that decide order of drawing, all objects get their own draw priority and after update all objects, we sort all objects based off of their priority, so we can draw objects based on order we want.

We also can specific object to another function by using getobject function.

- Turn Manager

In game, turn count and information whose turn is now should be kept track of by neutral service. So we make turn manager which save current turn and turn count. If we have to change turn and turn count we can use this component.

- Enemy Manager

When each room starts, we have to spawn enemies based on randomly chosen txt files, so enemy manager do this. And when these new enemies are spawned old enemies are have to be cleared also. This component also do this service. And when all enemies did their behavior we have to change turn to player. So when new enemies are spawned, this enemy manager have private vector of enemy pointer and push this new enemy to this vector and game object manager's vector both.

- Item Manager

We have two kinds of item – drop item and use item. And we have to keep track of these items. Drop items are made when enemies are defeated and use items are made when players entered shop. And player have to keep track of use items also, so when players get use item item manager give service that push this item to player's vector. And when players enter new place, abandoned items have to be destroyed. Item manager do this job by comparing player's use_items and this item manager's use_items.

- UI

UI also have to display this is whose turn and items that player get. By using turn manager and item manager UI effectively display these information to display.

- Audio

Some of objects have to play sound effects and some game phase have to play BGM, so Audio wrapper class do this functionalities.

4. Describe how physics works within your game, such as gravity and collision, etc.

-we make game based off of tile, so we give each object index. And we check collision based of index, and if two objects have same index, we considered these are collided.

5. Describe how AI works within your game, such as homing, flocking, movement patterns, etc.

- Enemies have to consider reachable indices based on their unique gimmick, and also have to move closet positions to player. So we converted indices to positions, and check distance between reachable indices and player's index. And we choose closet index.